PROJECT PROPOSAL & ENGINEERING BLUEPRINT

MiniSOC / ThreatLens

An AI-assisted security analyst layer that turns raw security telemetry into correlated, explainable attack incidents.

"Turning security alerts into understandable attack stories."

AI & Machine Learning | Cybersecurity | Open Innovation

EVENT
Presidential SPARKX Hackathon 2026

ORGANIZED BY
Presidential Software Club × Presidential Graduate School

DATES
17–18 September 2026 · 8:00 AM–5:00 PM

VENUE
Presidential Graduate School, Thapagaun, Kathmandu

TEAM NAME
[Team Name]

TEAM MEMBERS
[Name 1, Name 2, Name 3, Name 4]

TRACK
Cybersecurity

DOCUMENT VERSION
v1.0 — Single Source of Truth

Table of Contents

MiniSOC / ThreatLens — Project Proposal & Engineering Blueprint, v1.0

PART I — UNDERSTANDING THE PROJECT

1. The Project in Simple Words 5
2. The Problem 7
3. Existing Solutions & Why We Are Still Building This 9
4. Project Objectives 11

PART II — SYSTEM DESIGN & ARCHITECTURE

5. Complete System Architecture 13
6. Technology Stack 15
7. Agent Architecture 16
8. Event Normalization 18
9. Detection Engine 19
10. Correlation Engine 21
11. Incident Model 23
12. Risk / Severity Scoring 24
13. MITRE ATT&CK Mapping 25
14. AI Analyst 26
15. Response System 28
16. Frontend / Dashboard Architecture 29
17. API Design 30
18. Database Design 31
19. Repository Structure 32

PART III — BUILDING IT IN 24 HOURS

20. Development Workflow 34
21. Team of 4: Responsibilities 35
22. Git / Collaboration Workflow 36
23. Testing Strategy 37
24. Safe Cyber Range 38
25. Demo Scenario 39
26. Wazuh Comparison Strategy 40
27. What Makes This a Hackathon Project? 41
28. 24-Hour Engineering Roadmap 42
29. MVP Definition 43

PART IV — BEYOND THE HACKATHON

30. Future Roadmap 45
31. Security of Our Own System 46
32. Performance & Scalability 47
33. Project Risks 48
34. Backup Plan — The Demo Cannot Fail 49

PART V — APPROVAL & REFERENCE

35. Professor Approval Summary 51
36. Final Project Blueprint 53

PART I

Understanding the Project

What we are building, why it matters, what already exists, and what we are specifically setting out to solve in the next 24 hours.

01 The Project in Simple Words

02 The Problem

03 Existing Solutions & Why We Are Still Building This

04 Project Objectives

PART I · SECTION 01

The Project in Simple Words

> **KEY INSIGHT**
> A security tool that *collects* alerts is not the same as a security analyst who *understands* them. MiniSOC is the second thing.

The CCTV Analogy

Imagine a building with a hundred CCTV cameras. Every time something moves, a notification pops up: *"Motion detected at Camera 14."* "Motion detected at Camera 27." Over a busy hour, security staff might receive hundreds of these notifications. Almost all of them are nothing — a cat, a curtain, a passing car.

A trained security guard doesn't treat these as a hundred separate facts. They notice that Camera 14, then Camera 15, then the back-door sensor, then Camera 27 all triggered *in sequence, within two minutes, along one path through the building* — and they say: **"Someone walked from the car park to the loading bay. That's worth investigating."**

That sentence is the entire idea behind this project. Security tools (like Wazuh, a SIEM, or a log collector) are the cameras and the motion sensors. **MiniSOC is the guard who turns scattered notifications into one coherent story.**

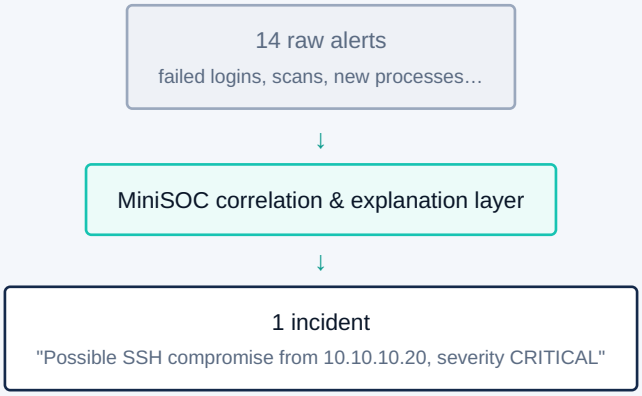


Fig. 1.1 — What MiniSOC does, in one picture.

Explaining It to Three Different People

To a Cybersecurity Professor

MiniSOC is a rule-based detection and event-correlation pipeline that ingests normalized security telemetry, groups related alerts into incidents using time/entity/sequence heuristics, maps

To a Non-Technical Judge

Instead of showing a security guard a hundred blinking lights, we show them one sentence: *"Someone tried to break into this server, they succeeded, and here's exactly what to do next."*

To a Beginner Developer

We receive JSON events over an API, run them through a handful of `if/then` rules, group the ones that look related, save the group as an "incident" in a database, and then ask an AI model to summarize that incident

the resulting attack chain to MITRE ATT&CK, and uses an LLM strictly as a post-hoc explanation and recommendation layer — not as the detector itself.

We prove it live by staging a safe, contained attack and watching the system explain it in real time.

in plain English. A React dashboard shows it all updating live.

What This Project Is *Not*

To avoid any confusion later in this document: we are not building a replacement for enterprise SIEM/SOC platforms such as Wazuh, Splunk, Microsoft Sentinel, or Elastic Security. Section 3 addresses this directly, because it is the single most likely question a professor or judge will ask.

PART I · SECTION 02

The Problem

Alert Volume Without Alert Meaning

Any machine connected to a network generates a constant stream of security events: login attempts, network connections, new processes, configuration changes. Even a small lab environment with two or three machines can produce dozens of log lines per minute. A real organization produces thousands.

This creates a well-documented problem in the security industry called **alert fatigue**: when analysts are shown too many low-context alerts, they either miss the important ones or stop trusting the system altogether.

An Alert

A single observation: *"Failed SSH login from 10.10.10.20 at 10:32:14."* True, but tells you almost nothing about whether you're under attack.

An Incident

"10.10.10.20 scanned this host, then attempted 14 SSH logins, succeeded, and ran a suspicious command — consistent with a brute-force compromise." Now you know what happened and what to do.

Why the Gap Exists

- **Volume:** too many individual events for a human to read one by one.
- **Context loss:** a single event, viewed alone, rarely reveals intent.
- **Time-correlation is manual work:** connecting "this happened, then that happened, four minutes later" across a log stream is exactly the kind of pattern-matching computers are good at and humans are slow at.
- **No dedicated analyst:** small organizations, student labs, and small businesses rarely have a 24/7 Security Operations Center (SOC) team to do this correlation manually.
- **Enterprise tooling is heavy:** full SIEM/SOC platforms are powerful but often require dedicated engineers to configure, tune, and operate effectively.

A Concrete Example

Here is a realistic sequence of raw events a monitored server might produce during an attack:

```
10:31 — Port scan detected from 10.10.10.20
10:32 — Multiple SSH connection attempts from 10.10.10.20
10:32 — 14 failed SSH logins for user 'admin'
10:33 — Successful SSH login for user 'admin' from 10.10.10.20
10:34 — Suspicious command executed: "wget http://... | sh"
```

Viewed as five separate log lines in a terminal, this is easy to miss during a busy shift, especially mixed in among hundreds of benign entries. MiniSOC's job is to turn exactly this kind of sequence into:

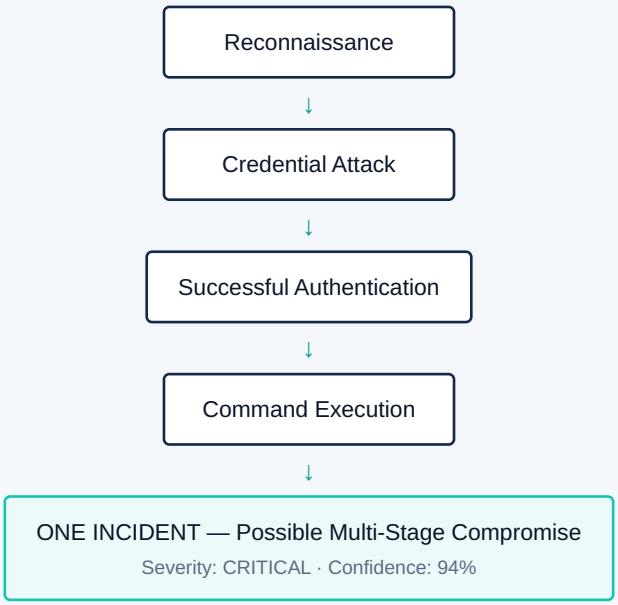


Fig. 2.1 — Five disconnected log lines become one attack story.

JUDGE QUESTION WE ARE ANSWERING

"Isn't this just what a SIEM already does?" — Partially, yes. Section 3 explains precisely where existing tools stop and where our project focuses its effort.

PART I · SECTION 03

Existing Solutions & Why We Are Still Building This

> **THE QUESTION EVERY JUDGE WILL ASK**
> "Wazuh, Splunk, Sentinel, and Elastic Security already exist — what exactly did you solve?" This section is our honest, rehearsed answer. Every team member should be able to recite the core distinction below.

What Mature SIEM/SOC Platforms Already Do Well

We are not claiming any of the following capabilities are missing from tools like Wazuh, Elastic Security, Microsoft Sentinel, or Splunk. They do these things well, at enterprise scale:

- Telemetry collection from a huge range of log sources and endpoints
- Log parsing, indexing, and long-term storage
- Rule-based and statistical detection engines
- Alerting and notification pipelines
- Dashboards, search, and threat-hunting interfaces
- MITRE ATT&CK integrations and threat intelligence feeds
- Endpoint monitoring agents

So Why Are We Building This?

Because **collecting and detecting** is a different problem from **correlating, contextualizing, and explaining**. Existing platforms give experienced analysts the raw material to do that correlation themselves — often through custom rules, saved searches, or paid add-on modules. Our project targets the specific, narrower gap:



Fig. 3.1 — The division of responsibility we are claiming.

Concretely, our differentiator is a lightweight, focused, analyst-oriented layer that can consume telemetry (optionally including telemetry a tool like Wazuh already collected) and produce:

- Attack stories — a human-readable narrative, not a log dump
- Incident timelines with clear start/end and progression
- Contextual severity that accounts for the whole sequence, not one event
- Evidence summaries an analyst can act on immediately
- MITRE ATT&CK technique chains derived from our own detection logic
- AI-assisted plain-language explanations and recommended next actions

Critically, MiniSOC is designed to **sit on top of** existing security tools conceptually, not replace them. A real deployment could receive events forwarded from Wazuh, Zeek, or plain system logs.

Comparison Table

Capability	Wazuh / Mature SIEM	MiniSOC
Collect telemetry	✓ Core strength	✓ Consumes it
Detect individual events	✓	✓ (deterministic rules)
Correlate into attack stages	Possible via custom rules/config	**Core focus**
Human-readable attack narrative	Limited by default	**Core feature**
MITRE ATT&CK mapping	✓	✓
AI-generated explanation	Not the primary focus	**Core feature**
Lightweight student/small-org deployment	Depends on setup	**Core goal**
Analyst workflow scope	Broad, general-purpose platform	Narrow, focused workflow

THE SENTENCE TO MEMORIZE

"Existing security tools are excellent at collecting telemetry and generating alerts. Our project focuses on what comes next: correlating those alerts into attack stories, prioritizing incidents, mapping attacker behavior to MITRE ATT&CK, and using AI to explain the incident and recommend the next action." — When asked "what did you actually solve?" the answer is: **"We reduced the gap between detection and understanding."**

PART I · SECTION 04

Project Objectives

1. Collect security telemetry from isolated lab endpoints.
2. Normalize different event formats into one common schema.
3. Detect suspicious behavior using deterministic, explainable rules.
4. Correlate related events across time, source, and target into incidents.
5. Generate clear incident timelines showing attack progression.
6. Map observed behavior to MITRE ATT&CK techniques.
7. Use AI to summarize and explain each incident in plain language.
8. Recommend concrete next actions for the analyst.
9. Provide a simple, real-time dashboard.
10. Demonstrate the full pipeline using a safe, isolated attack lab.

Priority Tiers

MUST-HAVE

- Event ingestion API
- 5–8 detection rules
- Correlation engine
- Incident creation & timeline
- MITRE mapping
- Dashboard with live alerts
- One rehearsed demo scenario

SHOULD-HAVE

- AI-generated incident summary
- Simulated response actions
- Confidence scoring
- Multiple attack scenarios

NICE-TO-HAVE

- Wazuh integration
- Zeek/Suricata telemetry
- Advanced process monitoring
- Polished animations

EXPLICITLY OUT OF SCOPE

Full enterprise SIEM features, Kubernetes, Kafka, an Elasticsearch cluster, a full SOAR platform, malware sandboxing, real firewall automation, and Windows telemetry. See Section 29 for the complete list.

PART II

System Design & Architecture

Every component of MiniSOC explained: what it does, why it exists, what technology it uses, and whether it is required for the 24-hour MVP.

05 Complete System Architecture
06 Technology Stack
07 Agent Architecture
08 Event Normalization
09 Detection Engine
10 Correlation Engine
11 Incident Model
12 Risk / Severity Scoring
13 MITRE ATT&CK Mapping
14 AI Analyst
15 Response System
16 Frontend / Dashboard Architecture
17 API Design
18 Database Design
19 Repository Structure

PART II · SECTION 05

Complete System Architecture

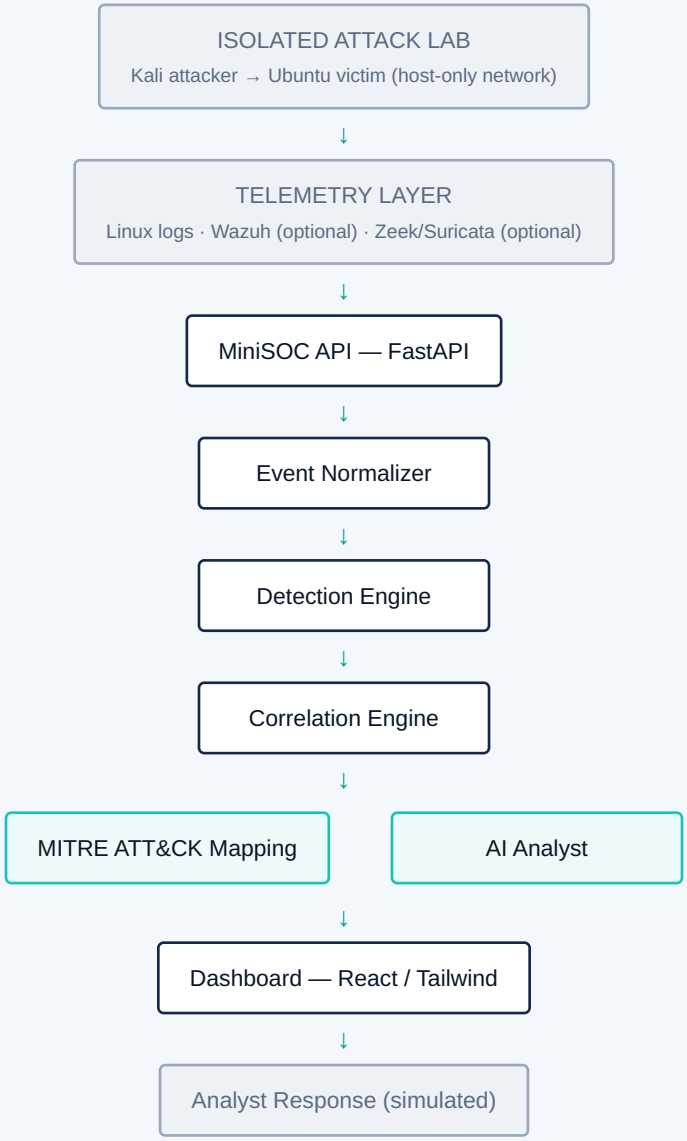


Fig. 5.1 — End-to-end MiniSOC pipeline.

Component Reference

Component	Input	Output	Tech	Required for MVP?
Attack Lab	Operator commands	Real network/auth activity	Kali + Ubuntu VMs	Yes
Agent	System/auth logs	Normalized JSON events	Python	Yes
MiniSOC API	HTTP POST events	Stored events, REST/ WebSocket data	FastAPI	Yes
Event Normalizer	Raw event formats	Canonical event schema	Python	Yes
Detection Engine	Normalized events	Alerts	Python rule engine	Yes
Correlation Engine	Alerts	Incidents	Python	Yes
MITRE Mapping	Rule/alert type	Technique IDs	Static lookup table	Yes
AI Analyst	Structured incident JSON	Summary + recommendations	LLM API	Should-have
Dashboard	REST + WebSocket data	Visual incidents/alerts	React + Tailwind	Yes
Response Actions	Analyst click	Simulated containment	Frontend + API flag	Should-have
Wazuh / Zeek / Suricata	Network/host activity	Additional telemetry	Third-party tools	Optional

WHAT HAPPENS IF A COMPONENT FAILS

Every layer is designed to degrade gracefully: if the AI Analyst API is unreachable, the incident still displays with its rule-based summary and MITRE mapping (see Section 34, Backup Plan). If Wazuh isn't wired up in time, the Python agent alone is sufficient to drive the entire demo.

PART II · SECTION 06

Technology Stack

Layer	Choice	Why
Backend API	Python + FastAPI	Rapid development, native async, easy WebSockets, huge security-tooling ecosystem, simple LLM API integration.
Database	SQLite	Zero configuration, file-based, perfectly sufficient for a 24-hour prototype and a handful of hosts. PostgreSQL is a documented future upgrade (Section 30).
Agent	Python	Cross-platform, trivial log parsing, simple HTTP client, no compilation step under time pressure.
Frontend	React + TypeScript + Tailwind CSS	Fast component iteration, strong ecosystem for dashboards, Tailwind removes CSS bikeshedding during a hackathon.
Charts	Recharts	Minimal setup, works well with React, sufficient for timelines and counts.
Real-time updates	WebSockets	Alerts should visibly appear live on the dashboard during the demo — polling would look static and unconvincing.
AI	LLM API (cloud or local, subject to availability)	Used only for explanation/summarization, never for primary detection (see callout below).
MITRE ATT&CK	Static JSON lookup table	Technique IDs are attached to detection rules, not invented dynamically.
Optional telemetry	Zeek / Suricata	Richer network visibility — explicitly optional, must never block the MVP.

WHY THIS TECHNOLOGY — THE DETECTION/AI SPLIT

```
Rules detect.
Correlation provides context.
AI explains.
```

The AI model is never asked to decide whether something is an attack. It only receives an already-structured incident and turns it into readable prose and recommendations. This keeps the system reliable, debuggable, and honest about what is deterministic versus generative.

PART II · SECTION 07

Agent Architecture

What Is the Agent?

A lightweight Python process running on the monitored (victim) machine. Its only job is to notice security-relevant activity, package it into a normalized JSON event, and send it to the MiniSOC API. It does not detect or decide anything — it observes and reports.

What It Collects (MVP Priority)

- SSH authentication events (success/failure) — from `/var/log/auth.log` or `journald`
- General system/service logs via `journald`
- Process execution events, if time allows
- Basic network connection information, if time allows

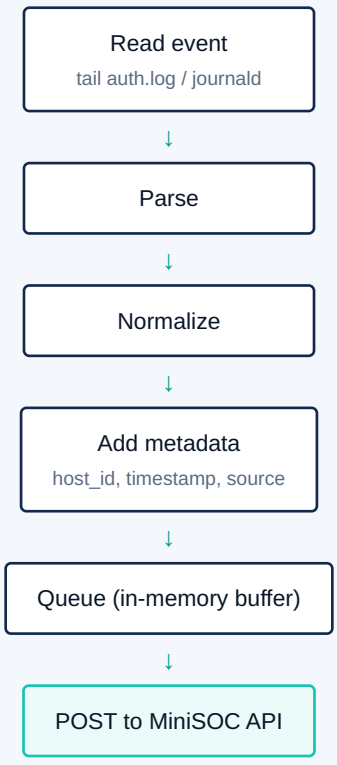


Fig. 7.1 — Agent internal pipeline.

Example Event Sent by the Agent

```
{
  "timestamp": "2026-09-17T10:31:24Z",
  "host": "ubuntu-server",
  "source_ip": "10.10.10.20",
  "event_type": "ssh_failed_login",
  "username": "admin",
  "severity": "low",
  "raw_event": "Failed password for admin from 10.10.10.20 port 51422 ssh2"
}
```

Operational Concerns (Keep Simple for MVP)

Concern	MVP Approach
Buffering	Small in-memory list; flush every N seconds or M events.
Retries	One retry with a short delay; drop and log on repeated failure — don't build a durable queue.
Duplicate events	Track last-seen log offset per file to avoid re-reading lines.
Host identity	Static hostname/IP configured at agent startup.
Authentication to API	Simple static API key in a request header — sufficient for a lab demo.
Server offline	Buffer in memory for the demo's short duration; no persistence needed.

> **MVP GUIDANCE**
>
> Do not build a production-grade agent. A single Python script that tails two log files and POSTs JSON is entirely sufficient and far more reliable under time pressure than a plugin-based collector framework.

PART II · SECTION 08

Event Normalization

Different tools produce wildly different event formats — a raw Linux log line looks nothing like a Wazuh JSON alert or a Zeek connection record. If the detection engine had to understand every possible input format, it would become unmaintainable within hours. Normalization solves this by converting everything into one canonical shape before anything else touches it.

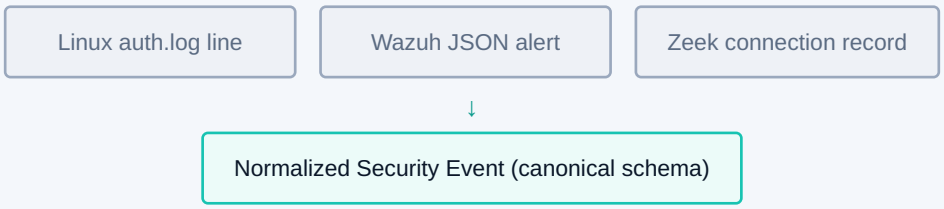


Fig. 8.1 — Everything becomes one shape before detection runs.

Canonical Event Schema

Field	Type	Required?	Notes
event_id	string (UUID)	Required	Generated on ingest
timestamp	ISO-8601 datetime	Required	UTC
host	string	Required	Reporting host identifier
source_ip	string	Optional	Origin of the activity, if known
destination_ip	string	Optional	Target of the activity, if known
event_type	enum string	Required	e.g. `ssh_failed_login`
severity	enum: low/medium/high	Required	Assigned at normalization time
username	string	Optional	Account involved, if any
process	string	Optional	Process name/command, if any
port	integer	Optional	
raw_event	string	Required	Original line, kept for evidence/audit
metadata	object	Optional	Free-form extra fields

PART II · SECTION 09

Detection Engine

The detection engine asks a narrow question about one event or a short window of events: *does this look suspicious on its own?* For a 24-hour build, deterministic rules are strongly preferred over machine learning — they are explainable, testable, and reliable during a live demo.

DON'T BUILD THIS

Do not train an ML classifier for core detection. It adds risk (data availability, training time, unpredictable output) without adding proportional value for a 24-hour scope. Five to eight well-chosen deterministic rules are more convincing to judges because you can explain exactly why each one fired.

MVP Rule Set

#	Rule	Logic	Severity	MITRE
1	SSH brute force	>5 failed logins within 60s from same source	High	T1110
2	Port scanning / recon	Connection attempts across multiple ports from one source in a short window	Medium	T1046
3	Successful login after failures	Success event follows ≥3 failures from same source within 5 min	High	T1078
4	Suspicious command execution	Known risky command pattern runs shortly after authentication	Critical	T1059
5	Unusual process activity	New/unexpected process spawned outside normal baseline window	Medium	T1059

Pseudocode — Rule 1 (SSH Brute Force)

```
FOR each incoming ssh_failed_login event:
    key = (source_ip, target_host)
    window = events_in_last_60_seconds(key, "ssh_failed_login")
    IF count(window) > 5:
        CREATE alert(
            type = "ssh_brute_force",
            severity = "high",
            mitre = "T1110",
            evidence = window
        )
```

For Every Rule, Define

- Inputs (which event types it watches)
- Logic and threshold
- Output (alert shape)
- Severity assigned
- MITRE technique attached
- Known false positives & how to test the rule

TESTING TIP

Each rule should be testable by sending a small, scripted burst of synthetic events directly to `POST /events` — no live attack required to verify detection logic (see Section 23).

PART II · SECTION 10

Correlation Engine

> **THIS IS THE CORE DIFFERENTIATOR**
>
> **Detection** says: "This event is suspicious." **Correlation** says: "These suspicious events are probably part of the same attack." Everything else in this project supports this one idea.

What the Correlation Engine Considers

- Same source IP?
- Same target host?
- Close together in time (within a configurable window, e.g. 5–10 minutes)?
- Do the event types form a logical attack sequence (recon → credential attack → auth → execution)?

MVP Algorithm

```
1. Group open alerts by (source_ip, target_host).
2. Sort the group by timestamp.
3. Compare the sequence of alert types against known attack-stage orderings.
4. Calculate a confidence score from: sequence match, time proximity,
   number of related alerts, and severity of each alert.
5. If an open incident already exists for this (source, target) pair,
   append the new alert to it and update last_seen; otherwise create
   a new incident.
```

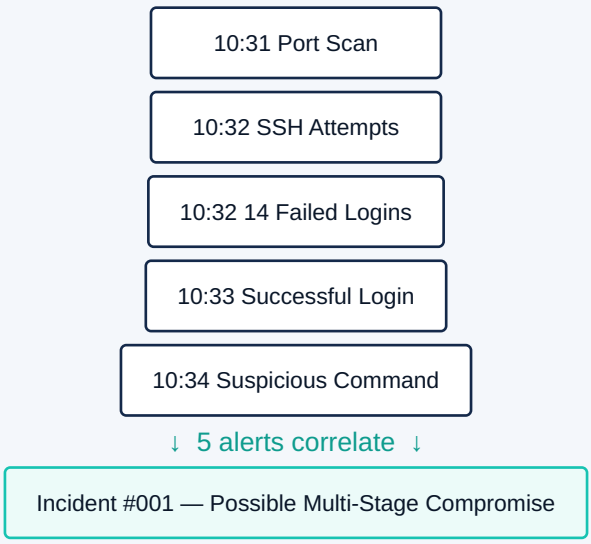


Fig. 10.1 — Five alerts collapse into one incident.

> **KEEP IT DETERMINISTIC**
>
> Confidence scoring can be a simple weighted sum (see Section 12) — resist the urge to use a probabilistic model here. Judges can follow a formula; they cannot follow a black box in 90 seconds.

PART II · SECTION 11

Incident Model

Incident Schema

Field	Type	Notes
incident_id	string (UUID)	Primary identifier
title	string	e.g. "Possible SSH Compromise"
severity	enum	low / medium / high / critical
confidence	float (0–1)	From the correlation engine (Section 12)
status	enum	See states below
first_seen / last_seen	datetime	Bounds of the attack window
source_entities	list	Source IP(s) involved
target_entities	list	Target host(s) involved
events	list of event_id	Underlying evidence
attack_chain	ordered list	Stage-by-stage progression
mitre_techniques	list	Technique IDs attached
ai_summary	text	Generated by the AI Analyst (Section 14)
recommendations	list	Suggested analyst actions

Incident States



Fig. 11.1 — Primary path. An incident may also be marked FALSE_POSITIVE from any state.

PART II · SECTION 12

Risk / Severity Scoring

Severity should be simple enough to compute by hand and explain to a judge in one sentence — not a hidden model output.

Scoring Inputs

- Base severity of each contributing alert (low=1, medium=2, high=3, critical=4)
- Number of related events in the incident
- Whether authentication succeeded (weight added if so)
- Whether a sensitive action occurred post-authentication (command execution, new process)
- How far along the expected attack progression the sequence reached

Example Calculation

```
score = sum(alert_base_severity)
      + (2 if successful_auth_after_failures else 0)
      + (3 if sensitive_action_post_auth else 0)
      + (1 per additional correlated alert beyond the first)

score  0-3   -> LOW
score  4-6   -> MEDIUM
score  7-9   -> HIGH
score  10+   -> CRITICAL
```

Incident	Contributing Factors	Score	Result
Single failed login	1 low alert	1	LOW
Brute force, no success	1 high alert (3) + 4 extra alerts (4)	7	HIGH
Full compromise chain	recon + brute force + success (+2) + command exec (+3)	12	CRITICAL

PART II · SECTION 13

MITRE ATT&CK Mapping

MITRE ATT&CK is a public, widely-used knowledge base of adversary tactics and techniques, organized as a matrix (e.g. Reconnaissance, Initial Access, Execution, Persistence…). Referencing it gives our incidents a vocabulary that any security professional immediately recognizes.

> **IMPORTANT DESIGN RULE**
>
> MITRE technique IDs are attached to our **detection rules**, not invented dynamically by the AI. The AI may explain a technique in plain language, but the mapping itself always comes from known, deterministic logic — this avoids hallucinated technique IDs.

Observed Behavior	MITRE Technique
Network / port scanning	T1046 — Network Service Discovery
Repeated failed logins (brute force)	T1110 — Brute Force
Successful login with valid credentials	T1078 — Valid Accounts
Command execution post-auth	T1059 — Command and Scripting Interpreter

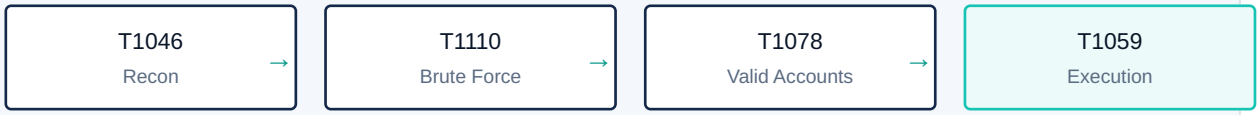


Fig. 13.1 — Example attack chain expressed in MITRE ATT&CK technique IDs.

PART II · SECTION 14

AI Analyst

WHERE AI ENTERS THE PIPELINE

AI is the last step, not the first. It receives an already-structured, already-correlated incident — it never sees raw logs and never decides whether something is an attack.

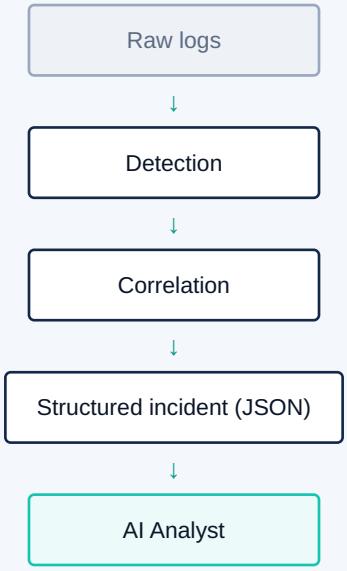


Fig. 14.1 — Correct pipeline. "Raw logs → AI decides everything" is explicitly rejected.

Example AI Input

```
{
  "incident_type": "Possible SSH compromise",
  "source": "10.10.10.20",
  "target": "ubuntu-server",
  "events": ["port_scan", "ssh_failed_login x14", "ssh_success", "suspicious_command"],
  "mitre": ["T1046", "T1110", "T1078", "T1059"],
  "severity": "critical"
}
```

Expected AI Output

- Executive summary (2–3 sentences)
- Attack explanation in plain language
- Evidence list (restated from structured input, not invented)
- Likely attack progression
- MITRE technique explanation
- Recommended investigation steps

- Recommended containment steps

Preventing Hallucination

- Only provide the AI with structured evidence — never raw, unfiltered logs
- Explicit system-prompt instruction: do not invent events not present in the input
- Clearly distinguish "evidence" fields from "inference/recommendation" fields in the prompt
- Label all AI recommendations as recommendations, never as actions already taken
- Never allow the AI to directly execute a containment action — a human always clicks the button

> **IMPORTANT SECURITY NOTE**
>
> Raw log content (`raw_event` fields) can contain attacker-controlled text. Sanitize or strip this field before it reaches the AI prompt to prevent prompt-injection via malicious log content (see Section 31).

PART II · SECTION 15

Response System

For the hackathon, response actions are **simulated**, not wired into real infrastructure. This is a deliberate scope decision, not a limitation to apologize for — connecting to a real firewall or EDR product is a production concern, not a 24-hour one.

MVP Actions (Simulated)

- Block Source IP
- Isolate Host
- Disable Account
- Mark Incident Contained

Future Production Integration

- Firewall API (block rule push)
- EDR agent isolation command
- SOAR platform playbook trigger
- Identity provider account lock

In the dashboard, clicking a response action updates the incident's `status` field and appends a response record — this is enough to demonstrate the full analyst workflow end-to-end.

PART II · SECTION 16

Frontend / Dashboard Architecture

Dashboard (Home)

- Alert / critical / incident / host counters
- Threat activity timeline chart
- Recent alerts list, color-coded by severity

Incident Page

- Title, severity, confidence
- Attack timeline & attack chain
- MITRE mapping
- AI analysis
- Recommended response buttons

Alert & Host Pages

- Alert: source, target, time, severity, rule, evidence
- Host: recent events, alerts, current status

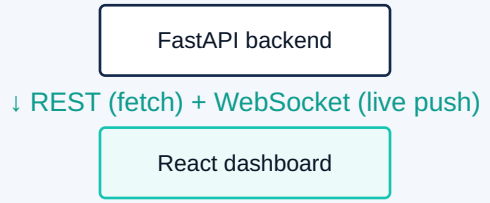


Fig. 16.1 — The dashboard reads state via REST on load, then patches it live via WebSocket.

PART II · SECTION 17

API Design

Endpoint	Purpose
`POST /events`	Agent submits a normalized event
`GET /events`	List recent events (debugging / host page)
`GET /alerts`	List alerts
`GET /alerts/{id}`	Alert detail
`GET /incidents`	List incidents
`GET /incidents/{id}`	Incident detail (timeline, MITRE, AI summary)
`GET /hosts`	List monitored hosts and status
`POST /incidents/{id}/analyze`	Trigger AI Analyst for an incident
`POST /incidents/{id}/response`	Record a simulated response action
`WS /ws/events`	Live push of new alerts/incidents to the dashboard

Example — POST /events

```
Request:
POST /events
Headers: X-Api-Key: <agent key>
Body:
{
  "timestamp": "2026-09-17T10:32:14Z",
  "host": "ubuntu-server",
  "source_ip": "10.10.10.20",
  "event_type": "ssh_failed_login",
  "severity": "low",
  "raw_event": "..."
}

Response: 201 Created
{ "event_id": "8f2c...", "status": "stored" }
```

Authentication for the MVP: a single static API key checked via request header, sufficient for an isolated lab demo (see Section 31 for hardening notes beyond the MVP).

PART II · SECTION 18

Database Design

SQLite is enough for the hackathon: a handful of hosts, at most a few thousand events, and a single FastAPI process. No server process to run, no connection pool to configure — just a file.

Tables

Table	Key Columns
hosts	host_id (PK), hostname, ip, status, last_seen
events	event_id (PK), host_id (FK), timestamp, event_type, source_ip, severity, raw_event
alerts	alert_id (PK), rule_name, severity, mitre_id, created_at
incident_events	incident_id (FK), event_id (FK) — join table
incidents	incident_id (PK), title, severity, confidence, status, first_seen, last_seen, ai_summary
mitre_techniques	technique_id (PK), name, tactic
responses	response_id (PK), incident_id (FK), action, timestamp

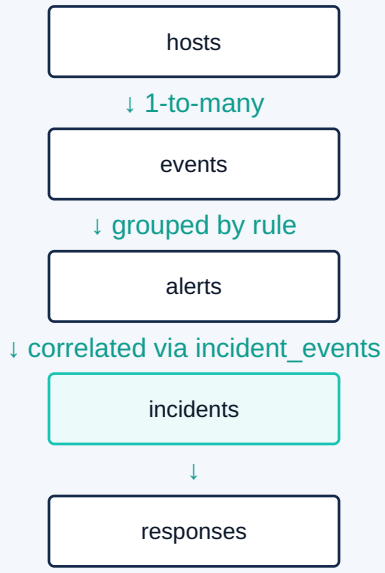


Fig. 18.1 — Simplified entity relationship flow. incidents also reference mitre_techniques (many-to-many).

PART II · SECTION 19

Repository Structure

```
minisoc/
├─ backend/
│   ├─ app/
│   │   ├─ api/          # FastAPI routers
│   │   ├─ models/       # DB models
│   │   ├─ schemas/      # Pydantic schemas
│   │   ├─ services/     # shared business logic
│   │   ├─ detection/    # rule engine
│   │   ├─ correlation/  # correlation engine
│   │   ├─ mitre/        # technique lookup table
│   │   ├─ ai/           # AI analyst integration
│   │   └─ main.py
│   └─ tests/
├─ agent/
│   ├─ collectors/       # log tailers
│   ├─ parsers/
│   ├─ normalizer/
│   └─ agent.py
├─ frontend/
│   └─ src/
│       ├─ components/
│       ├─ pages/
│       ├─ services/
│       └─ hooks/
├─ attack-lab/
│   ├─ scenarios/
│   └─ README.md
├─ docs/
├─ docker-compose.yml
├─ README.md
└─ .gitignore
```

Directory	Purpose
backend/app/detection	One file per rule family (ssh.py, network.py, process.py)
backend/app/correlation	Grouping, sequencing, and confidence-scoring logic
agent/	Standalone script deployed to the victim VM
attack-lab/scenarios	Scripted attack sequences used for the live demo
docs/	This document and any supporting diagrams

PART III

Building It in 24 Hours

Team structure, workflow, testing, the cyber range, and an hour-by-hour roadmap for turning this document into a working demo.

20 Development Workflow

21 Team of 4: Responsibilities

22 Git / Collaboration Workflow

23 Testing Strategy

24 Safe Cyber Range

25 Demo Scenario

26 Wazuh Comparison Strategy

27 What Makes This a Hackathon Project?

28 24-Hour Engineering Roadmap

29 MVP Definition

PART III · SECTION 20

Development Workflow

Build in this order. Each phase should produce something runnable before the next phase begins — this keeps the team always sitting on a working (if incomplete) system.

Phase	Deliverable
1. Architecture + schemas	Repo skeleton, event/incident schemas agreed on paper
2. Backend API	FastAPI skeleton with `/events` working end-to-end
3. Database	SQLite tables created, events persist
4. Agent	Python agent tails logs, posts real events
5. Detection engine	First 2–3 rules produce alerts from synthetic events
6. Correlation engine	Related alerts merge into one incident
7. MITRE mapping	Alerts carry technique IDs
8. Frontend	Dashboard renders alerts/incidents from the API
9. AI	Incident JSON → AI summary wired in
10. Real-time updates	WebSocket pushes new alerts live to the dashboard
11. Attack lab	Kali/Ubuntu VMs produce real telemetry on demand
12. Integration	Full pipeline runs against the real attack lab
13. Testing	Test matrix from Section 23 passes
14. Demo preparation	Script rehearsed twice, backup plan ready

> **RULE OF THUMB**
>
> Do not start the frontend before `POST /events` and at least one detection rule work end-to-end. A beautiful dashboard with no real data behind it wastes hours and impresses no one.

PART III · SECTION 21

Team of 4: Responsibilities

Developer 1 — Backend / Security Logic

- FastAPI application & routing
- Database models & SQLite setup
- Detection engine (rules)
- Correlation engine

Developer 2 — Frontend

- React app structure & routing
- Dashboard, incident, alert, host pages
- Charts (Recharts)
- WebSocket client integration

Developer 3 — Agent / Cyber Range

- Ubuntu victim VM setup
- Python agent (log tailing, normalization, POST)
- Kali attacker VM & scripted attack scenarios
- Wazuh integration, if time allows

Developer 4 — AI / MITRE / Integration

- AI Analyst prompt design & API wiring
- MITRE technique lookup table
- End-to-end integration testing
- Documentation & demo script

Dependency Notes

- Dev 2 (Frontend) is blocked on Dev 1 shipping a minimal `/alerts` and `/incidents` endpoint — agree on the JSON shape on paper in Hour 0–2 so both can work in parallel against a shared mock.
- Dev 4's AI integration is blocked on Dev 1's incident schema being stable — freeze that schema early.
- Dev 3's attack lab can be built entirely independently and in parallel from Hour 0.

PART III · SECTION 22

Git / Collaboration Workflow

- **main** branch always stays runnable — never commit broken code directly to it.
- One short-lived feature branch per person/task: `feature/detection-rules`, `feature/dashboard-incident-page`.
- Commit early and often with plain descriptive messages (`add ssh brute-force rule`, not `wip`).
- Small, frequent pull requests merged by whoever is free — do not wait for a formal review process in a 24-hour event.
- Agree on the shared schemas (Sections 8, 11, 18) before writing code to minimize merge conflicts between backend and frontend work.
- Use a `.env` file (git-ignored) for the AI API key and any secrets — never commit credentials.
- Keep a running `README.md` with setup steps — you will need it again at 6 a.m.
- Track remaining work with GitHub Issues or a shared checklist, not a Discord message that scrolls away.

> **DON'T OVERCOMPLICATE THIS**
>
> No release branching strategy, no signed commits, no CI pipeline required. The only goal is: nobody blocks anybody, and main always runs.

PART III · SECTION 23

Testing Strategy

Test each layer independently before testing the full pipeline — this makes it far faster to find where something broke.

Layer	How to Test
Agent	Manually append a fake line to the watched log file and confirm the agent picks it up.
API	POST a synthetic event with curl/Postman; confirm 201 and DB row.
Detection	Send 10 scripted failed-login events; confirm exactly one brute-force alert fires.
Correlation	Send a scripted event sequence (recon → brute force → success → command); confirm one incident, not four.
MITRE	Confirm each rule's alert carries the expected technique ID.
AI	Feed a fixed sample incident JSON; confirm the response has all required fields.
Frontend	Confirm a manually-triggered alert appears on the dashboard without a page refresh.
End-to-end	Run the full scripted attack scenario from a clean state and confirm the entire pipeline lights up correctly.

KEY INSIGHT

You do not need a live exploit to test 90% of this system. Scripted synthetic events sent directly to the API validate detection and correlation logic far faster and more reliably than repeatedly running a real attack.

PART III · SECTION 24

Safe Cyber Range

IMPORTANT SECURITY NOTE

All attack demonstrations occur inside a fully isolated virtual network. At no point does any activity touch the Presidential Graduate School network or any real-world system. This document deliberately does not provide real-world attack instructions.

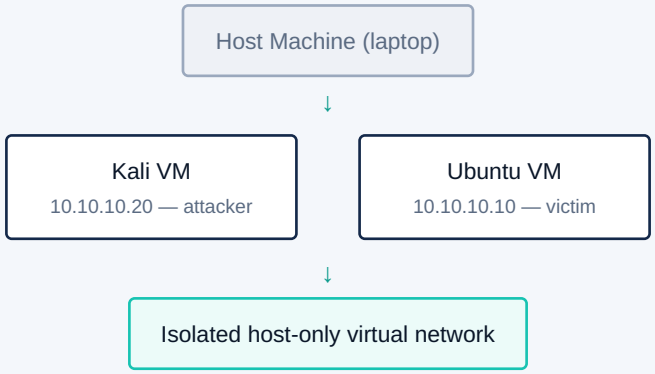


Fig. 24.1 — The entire attack lab runs on one laptop, on a virtual network with no route to the internet or campus network.

Why This Matters

- A host-only (or internal) virtual network has no bridge to the real college network — the attacker VM cannot reach anything outside the pair.
- The MiniSOC backend can run on the host machine or a third VM, also inside the same isolated network segment.
- This gives the team a fully controlled, repeatable environment to demonstrate real attack telemetry without any risk to shared infrastructure.

PART III · SECTION 25

Demo Scenario

The exact story to tell the judges, step by step:

1. Show the dashboard at rest: `0 Critical · 0 Incidents · 3 Hosts · STATUS: GREEN`.
2. Announce: "We're going to simulate an attack inside an isolated lab."
3. Run the scripted attack scenario against the Ubuntu victim VM.
4. The agent picks up authentication and command telemetry in real time.
5. MiniSOC's detection engine fires alerts as they occur — visible live on screen.
6. Alerts appear on the dashboard via WebSocket, one by one.
7. The correlation engine groups them into a single incident.
8. MITRE ATT&CK technique chips appear on the incident page.
9. Click "Analyze" — the AI Analyst generates the plain-language summary and recommendations.
10. The analyst (presenter) reviews the recommended response.
11. Click "Simulate Block IP."
12. Incident status changes to `CONTAINED`.

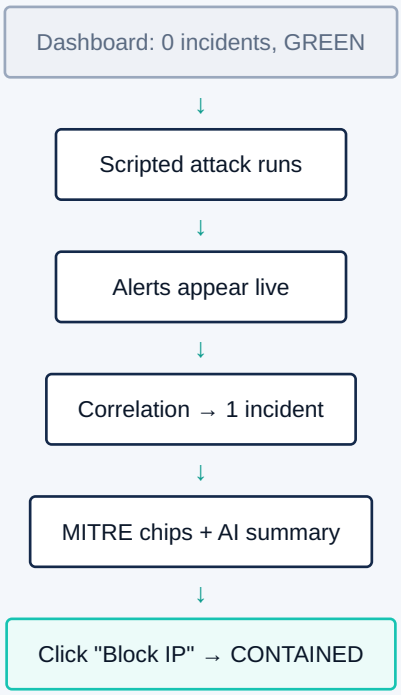


Fig. 25.1 — The full demo arc, roughly 90 seconds end to end.

REHEARSE THIS TWICE

Time the full demo before presenting. If it runs long, cut narration, not steps — the live progression from alerts to incident to AI explanation to response is the entire point.

PART III · SECTION 26

Wazuh Comparison Strategy

NEVER SAY THIS

"Wazuh can't do this." If Wazuh actually can do something and a judge knows Wazuh, that claim will be challenged immediately and will hurt the team's credibility for the rest of the pitch.

SAY THIS INSTEAD

"We're using Wazuh (or plain system logs) as our telemetry reference. Our goal isn't to replace Wazuh — we're demonstrating a different layer of the workflow: incident correlation, MITRE-mapped attack storytelling, and AI-assisted explanation."

Side-by-Side Talking Points

Capability	Wazuh	MiniSOC
Telemetry	✓	✓ (consumes it)
Detection	✓	✓
Alerting	✓	✓
Correlation	Existing capability / custom rules	**Core focus**
Incident timeline	Limited by default	**Core feature**
Attack story	Not primary focus	**Core feature**
MITRE	✓	✓
AI explanation	Not the focus	**Core feature**
Lightweight, focused workflow	Broad, general platform	**Core goal**

Be honest about overlap where it exists. The credibility of the pitch comes from precision about the gap we are filling, not from overstating the competition.

PART III · SECTION 27

What Makes This a Hackathon Project?

Strengths for a 24-Hour Format

- Clear, achievable MVP with a hard scope line
- Strong, obvious cybersecurity relevance
- A genuinely visible, live demo (not a static slide)
- Uses existing technologies intelligently rather than reinventing them
- No model training required
- Modular architecture — components can fail independently without sinking the whole demo
- Real room to extend after the hackathon (Section 30)

Honest Limitations

- Detection rule set is intentionally small (5–8 rules)
- Correlation logic is heuristic, not formally validated
- Single-host SQLite storage, not built for scale
- AI recommendations require human review before acting
- Response actions are simulated, not connected to real infrastructure

PART III · SECTION 28

24-Hour Engineering Roadmap

Hours	Focus	End-of-Phase State
0–2	Architecture, repo, schemas	Repo skeleton pushed; event/incident schemas agreed on paper
2–6	Backend + database	`POST /events` stores rows in SQLite
6–9	Detection engine	3+ rules produce alerts from synthetic events
9–12	Agent + telemetry	Real Ubuntu VM events flow into the API
12–15	Attack lab + scenarios	Scripted attack sequence runs end to end
15–19	Frontend	Dashboard, incident, alert, host pages render real data
19–21	Correlation + MITRE + AI	One incident with technique chips and an AI summary
21–23	Integration + testing	Full test matrix (Section 23) passes twice in a row
23–24	Demo + pitch + backup	Demo rehearsed, backup video and sample data ready

If Behind Schedule — Cut First

- Wazuh/Zeek/Suricata integration (use the plain agent only)
- Simulated response buttons (keep incident view read-only)
- Extra detection rules beyond the first 3–4
- Chart polish — a counter and a list beat an empty chart

If Ahead of Schedule — Add

- A second, different attack scenario for variety
- Confidence-score visualization on the incident page
- Host status page polish
- Zeek/Suricata for richer network evidence

PRIORITIZE RELIABILITY OVER FEATURES

A three-rule pipeline that works flawlessly live beats an eight-rule pipeline that might crash mid-demo. When in doubt, cut scope, not stability.

PART III · SECTION 29

MVP Definition

MUST WORK

- ☐ Agent collecting real Linux telemetry
- ☐ Ingestion API (`POST /events`)
- ☐ SQLite storage
- ☐ 3–5 detection rules
- ☐ Correlation engine
- ☐ Incident creation & timeline
- ☐ Dashboard with live alerts
- ☐ MITRE mapping
- ☐ AI explanation
- ☐ One rehearsed demo scenario

OPTIONAL

- ☐ Zeek
- ☐ Suricata
- ☐ Advanced process monitoring
- ☐ More sophisticated risk scoring
- ☐ Real response automation

DO NOT BUILD

- ✗ Full enterprise SIEM
- ✗ Kubernetes
- ✗ Kafka
- ✗ Elasticsearch cluster
- ✗ Full SOAR platform
- ✗ Malware sandbox
- ✗ Windows agent (unless everything else is complete)

PART IV

Beyond the Hackathon

Where this could go next, how we'd secure it for real use, what could break during the demo, and how we make sure the demo cannot fail.

30 Future Roadmap

31 Security of Our Own System

32 Performance & Scalability

33 Project Risks

34 Backup Plan — The Demo Cannot Fail

PART IV · SECTION 30

Future Roadmap

Version	Focus
V1 (hackathon)	Linux-only MVP, isolated lab, deterministic rules
V2	Windows telemetry support
V3	Zeek / Suricata network telemetry integration
V4	Multiple simultaneous data sources
V5	Threat intelligence feed integration
V6	More advanced correlation (graph-based, cross-host)
V7	Real automated response (firewall/EDR integration)
V8	Multi-tenant cloud deployment

V1–V3 are realistic near-term extensions for a student team. V6–V8 represent genuine production engineering effort and are explicitly future work, not something to attempt to sketch out during the hackathon itself.

PART IV · SECTION 31

Security of Our Own System

A security project should hold itself to a basic security standard, even under hackathon time pressure.

Concern	MVP Approach
API authentication	Static API key per agent, checked on every request
Input validation	Pydantic schemas reject malformed events at the API boundary
Rate limiting	Basic per-key request cap to prevent accidental flooding
Secrets management	`.env` file, git-ignored, never committed
Least privilege	Agent only reads specific log files, no root shell access required
Agent authentication	Same static API key mechanism as above
Secure communication	HTTPS/TLS in any non-local deployment; plain HTTP acceptable for an isolated local demo
Sanitizing AI inputs	Strip/limit raw_event content before including it in an AI prompt
Prompt injection defense	Treat all log-derived text as untrusted data, never as instructions, in the AI system prompt
Audit logs	Every response action is timestamped and recorded in the `responses` table

PART IV · SECTION 32

Performance & Scalability

Expected MVP Scale

```
1-10 monitored hosts
Hundreds to low thousands of events during the demo
SQLite, single FastAPI process
```

This is comfortably within SQLite and a single-process FastAPI server's capabilities — no separate scaling work is needed for the hackathon.

What Would Change at Larger Scale

Scale	What Would Need to Change
~100 hosts	Move from SQLite to PostgreSQL; add basic connection pooling
~1,000 hosts	Introduce a message queue (e.g. Redis streams) between agents and the API; move detection/correlation to background workers
~10,000 hosts	Distributed event storage (Elasticsearch/OpenSearch), horizontally scaled workers, dedicated ingestion tier

> **NOT REQUIRED FOR THE HACKATHON**
>
> None of the scaling work above should be attempted in the 24-hour window. It is documented here to show the architecture was designed with a believable growth path, not to be built now.

PART IV · SECTION 33

Project Risks

Risk	Probability	Impact	Mitigation
Agent fails to collect logs	Medium	High	Test agent against real VM early (Phase 4); keep log-parsing logic minimal
Wazuh integration takes too long	High	Low	Treat as optional from the start; plain agent alone drives the full demo
AI API unavailable during demo	Medium	Medium	Pre-generate a sample AI response as fallback (Section 34)
WebSocket instability	Low	Medium	Fall back to short-interval polling if needed
False positives during live demo	Medium	Low	Use a scripted, rehearsed attack scenario rather than improvising
Correlation engine doesn't trigger	Medium	High	Unit-test correlation against synthetic event sequences before integration
Frontend incomplete at deadline	Medium	High	Build dashboard against a mock API from Hour 2, so it's never blocked
Live attack demo fails on stage	Medium	High	Pre-recorded backup video (Section 34)
Team merge conflicts	Low	Medium	Small, frequent commits; schemas frozen early (Section 22)
Time runs out before integration	Medium	High	Strict phase gating (Section 20); cut scope per Section 28's contingency plan

PART IV · SECTION 34

Backup Plan — The Demo Cannot Fail

A live demo depending on real-time VM networking, a live LLM API call, and a WebSocket connection has several points that could hiccup in front of judges. We prepare fallbacks for all of them in advance.

What We Prepare in Advance

- A pre-recorded backup video of a full successful demo run
- Static sample telemetry (a JSON file of a full event sequence)
- A sample incident JSON, already correlated and MITRE-mapped
- Sample dashboard data, loadable without the live pipeline
- An offline/pre-generated AI response example for the same incident

If Something Fails Live

- Internet/AI API down → load the pre-generated AI response for the same incident
- VM networking issue → replay the static sample telemetry file directly into the API
- Dashboard/WebSocket issue → switch to the backup video mid-pitch and narrate over it
- Total failure → the backup video alone tells the complete story

RULE

Every live element has an offline equivalent ready before the team walks on stage. Nothing in the pitch should depend on a single point of failure with no fallback.

PART V

Approval & Reference

A concise, submittable proposal summary, followed by a single-page cheat sheet the whole team can keep open while building.

35 Professor Approval Summary

36 Final Project Blueprint

PART V · SECTION 35

Professor Approval Summary

Project Title

MiniSOC / ThreatLens — An AI-Assisted Security Analyst Layer

Problem

Security telemetry is abundant, but raw alerts lack context. Analysts — especially in small organizations and student environments without a dedicated SOC — struggle to connect related events into a coherent understanding of what actually happened.

Proposed Solution

A lightweight pipeline that normalizes telemetry, detects suspicious events with deterministic rules, correlates related alerts into incidents, maps behavior to MITRE ATT&CK, and uses AI strictly to explain the already-structured incident and recommend next steps.

Objectives

Collect → Normalize → Detect → Correlate → Map to MITRE → Explain with AI → Recommend a response — demonstrated live against a safe, isolated attack lab.

Innovation

The contribution is not a new detection algorithm; it is a focused, analyst-oriented correlation and explanation layer designed to sit on top of existing telemetry sources, closing the gap between "alert" and "understood incident" for teams without dedicated SOC analysts.

Technology Stack

Python, FastAPI, SQLite, React, TypeScript, Tailwind CSS, WebSockets, a static MITRE ATT&CK lookup table, and an LLM API for explanation only.

Architecture

Isolated attack lab → telemetry layer → MiniSOC API → normalizer → detection engine → correlation engine → MITRE mapping + AI analyst → dashboard → simulated analyst response (full diagram in Section 5).

Expected Output

A working demonstration in which a scripted, contained attack is detected, correlated into one incident, explained in plain language, and responded to — live, on a dashboard, within roughly 90 seconds.

Demo Plan

See Section 25 for the full script; a pre-recorded backup and static sample data (Section 34) ensure the demonstration can proceed even if a live component fails.

Limitations

Linux-only telemetry, a small deterministic rule set, heuristic (not formally validated) correlation logic, single-host SQLite storage, and simulated — not real — response actions. These are documented scope decisions appropriate to a 24-hour build, not oversights.

Future Scope

Windows telemetry, Zeek/Suricata network data, threat intelligence feeds, graph-based cross-host correlation, real response automation, and multi-tenant deployment (full roadmap in Section 30).

PART V · SECTION 36

Final Project Blueprint

One page. Print it, pin it, keep it open on a second monitor.

PROJECT	MiniSOC / ThreatLens
PROBLEM	Alert overload + lack of contextual incident understanding
SOLUTION	Analyst-focused correlation and AI explanation layer
INPUT	Security telemetry (Linux auth/system logs, optionally Wazuh/Zeek/Suricata)
PROCESS	Normalize → Detect → Correlate → Map to MITRE → Explain → Recommend
OUTPUT	One actionable, explained incident instead of many disconnected alerts
STACK	Python · FastAPI · SQLite · React · TypeScript · Tailwind · WebSockets · MITRE ATT&CK · LLM API
MVP	Linux telemetry + isolated attack lab, 3–5 detection rules, correlation, MITRE mapping, AI summary, live dashboard
DEMO	Controlled attack → telemetry → live alerts → correlation → attack story → AI analysis → simulated response
NOT	A Wazuh/SIEM replacement — a correlation & explanation layer that sits on top of existing telemetry

THE ONE SENTENCE

"Existing security tools collect and detect. MiniSOC correlates, contextualizes, explains, and prioritizes — reducing the gap between detection and understanding."